



深度学习与计算机视觉

Pytorch入门-基本数据类型与操作

教师：王一帆

pytorch

- PyTorch: 由Facebook的人工智能研究团队开发的开源深度学习框架
- 2016年, Facebook的AI研究团队 (FAIR) 公开了PyTorch, 其旨在提供一个快速, 灵活且动态的深度学习框架。
- PyTorch的设计哲学与Python的设计哲学非常相似: 易读性、简洁性优于隐式的复杂性。
- PyTorch用Python语言编写, 是Python的一种扩展, 这使得其更易于学习和使用。





pytorch安装

新建或激活虚拟环境（此步可选）：

`conda create -n env_name python=3.8`（新建）

`conda activate env_name`（激活）

env_name: 一个由自己命名的虚拟环境名字

pytorch安装

进入pytorch官网，选择适合自己的版本：

<https://pytorch.org/get-started/locally/>

Select your preferences and run the install command. Stable represents the most currently tested and supported version of PyTorch. This should be suitable for many users. Preview is available if you want the latest, not fully tested and supported, builds that are generated nightly. Please ensure that you have **met the prerequisites below (e.g., numpy)**, depending on your package manager. Anaconda is our recommended package manager since it installs all dependencies. You can also **install previous versions of PyTorch**. Note that LibTorch is only available for C++.

PyTorch Build	Stable (2.1.0)	Preview (Nightly)		
Your OS	Linux	Mac	Windows	
Package	Conda	Pip	LibTorch	Source
Language	Python	C++ / Java		
Compute Platform	CUDA 11.8	CUDA 12.1	ROCm 5.6	CPU
Run this Command:	<pre>pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118</pre>			

选择合适版本并按命令安装

NOTE: PyTorch LTS has been deprecated. For more information, see [this blog](#).

pytorch安装

命令行执行对应的安装命令

```
pip3 install torch torchvision torchaudio --index-url  
https://download.pytorch.org/whl/cu118
```

```
Collecting certifi>=2017.4.17 (from requests->torchvision)  
  Downloading https://download.pytorch.org/whl/certifi-2022.12.7-py3-none-any.whl (155 kB)  
    155.3/155.3 kB 9.7 MB/s eta 0:00:00  
Collecting mpmath>=0.19 (from sympy->torch)  
  Downloading https://download.pytorch.org/whl/mpmath-1.3.0-py3-none-any.whl (536 kB)  
    536.2/536.2 kB 16.4 MB/s eta 0:00:00  
Installing collected packages: mpmath, urllib3, typing-extensions, sympy, networkx, MarkupSafe, idna, fsspec, filelock, charset-normalizer, certifi, request  
s, jinja2, torch, torchvision, torchaudio  
Successfully installed MarkupSafe-2.1.2 certifi-2022.12.7 charset-normalizer-2.1.1 filelock-3.9.0 fsspec-2023.4.0 idna-3.4 jinja2-3.1.2 mpmath-1.3.0 network  
x-3.0 requests-2.28.1 sympy-1.12 torch-2.1.0+cu118 torchaudio-2.1.0+cu118 torchvision-0.16.0+cu118 typing-extensions-4.4.0 urllib3-1.26.13
```

```
(cv2) C:\Users\zwb12>python  
Python 3.11.5 | packaged by Anaconda, Inc. | (main, Sep 11 2023, 13:26:23) [MSC v.1916 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license" for more information.  
>>> import torch  
>>> torch.cuda.is_available()  
True  
>>> |
```

表示安装成功

pytorch安装

或者不指定版本直接执行命令

```
pip install torch torchvision
```

会根据你的机器自动安装pytorch和torchvision

■ N维数组 (ndarrays)

N维数组是机器学习和神经网络的主要数据结构

0-d (标量)



1.0

一个类别

1-d (向量)



[1.0, 2.7, 3.4]

一个特征向量

2-d (矩阵)

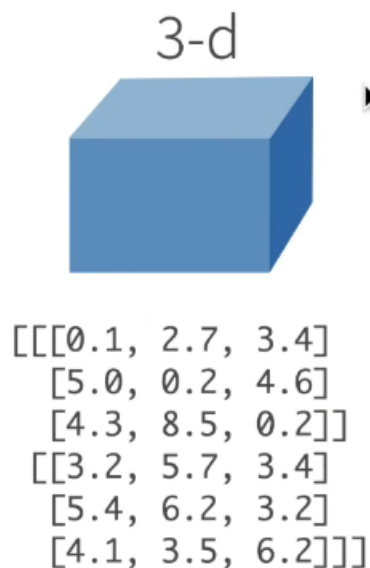


[[1.0, 2.7, 3.4]
[5.0, 0.2, 4.6]
[4.3, 8.5, 0.2]]

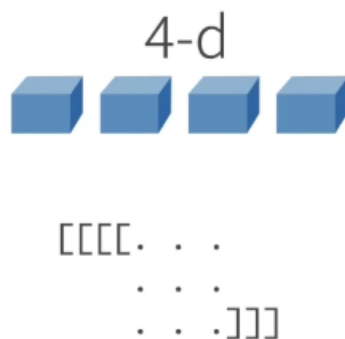
一个样本—特征矩阵

■ N维数组

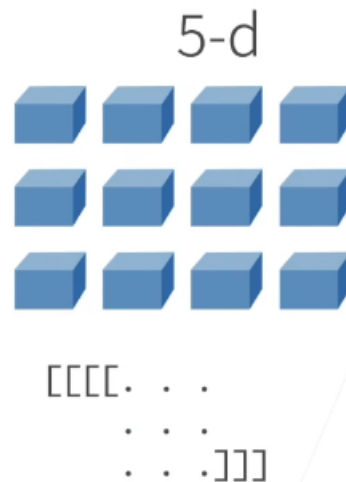
N维数组是机器学习和神经网络的主要数据结构



RGB图片 (宽x
高x通道)



一个RGB图片
批量 (批量大小
x宽x高x通道)



一个视频批量 (批量大
小x时间x宽x高x通道)

■ 基本数据：Tensor

- Tensor (张量), 是PyTorch中的基本操作对象, 可以看做是包含单一数据类型元素的多维矩阵。
- 从使用角度来看, Tensor与NumPy的ndarray非常类似, 相互之间也可以自由转换, 只不过Tensor还支持GPU的加速。

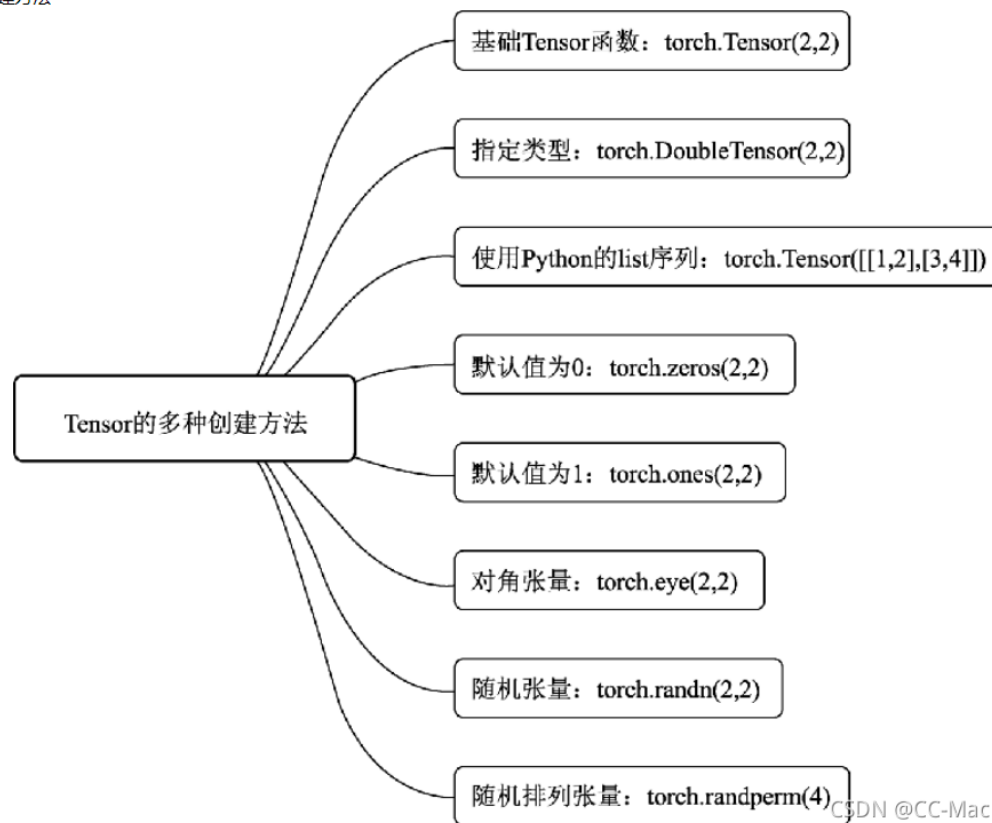
数据类型 [Ⓜ]	CPU Tensor [Ⓜ]	GPU Tensor [Ⓜ]
32位浮点 [Ⓜ]	<code>torch.FloatTensor[Ⓜ]</code>	<code>torch.cuda.FloatTensor[Ⓜ]</code>
64位浮点 [Ⓜ]	<code>torch.DoubleTensor[Ⓜ]</code>	<code>torch.cuda.DoubleTensor[Ⓜ]</code>
16位半精度浮点 [Ⓜ]	N/A [Ⓜ]	<code>torch.cuda.HalfTensor[Ⓜ]</code>
8位无符号整型 [Ⓜ]	<code>torch.ByteTensor[Ⓜ]</code>	<code>torch.cuda.ByteTensor[Ⓜ]</code>
8位有符号整型 [Ⓜ]	<code>torch.CharTensor[Ⓜ]</code>	<code>torch.cuda.CharTensor[Ⓜ]</code>
16位有符号整型 [Ⓜ]	<code>torch.ShortTensor[Ⓜ]</code>	<code>torch.cuda.ShortTensor[Ⓜ]</code>
32位有符号整型 [Ⓜ]	<code>torch.IntTensor[Ⓜ]</code>	<code>torch.cuda.IntTensor[Ⓜ]</code>
64位有符号整型 [Ⓜ]	<code>torch.LongTensor[Ⓜ]</code>	<code>torch.cuda.LongTensor[Ⓜ]</code>

CSDN @CC-Mac

■ Tensor的创建

- 形状
- 元素数据类型
- 元素的值

Tensor的多种创建方法



CSDN @CC-Mac



■ torch.tensor

- 通过提供包含数值的python列表为所需张量中每个元素赋值

```
1 import torch
2
3 a = torch.tensor([[2, 3, 4, 5],
4                  [1, 2, 3, 4]])
5 print(a)
```

```
tensor([[2, 3, 4, 5],
        [1, 2, 3, 4]])
```

进程已结束,退出代码0

■ torch.FloatTensor

- 用于创建数据类型为浮点型的Tensor
- 传递给torch.FloatTensor的参数可以是列表，也可以是一个维度值

```
1 import torch
2
3 a = torch.FloatTensor(2, 3)
4 b = torch.FloatTensor([2, 3, 4, 5])
5 print(a)
6 print(b)
```

```
tensor([[ -9.4497e+21,  1.1701e-42,  0.0000e+00],
        [ 0.0000e+00,  0.0000e+00,  0.0000e+00]])
tensor([2., 3., 4., 5.])
```

进程已结束,退出代码0



■ torch.IntTensor

- torch.IntTensor用于生成数据类型为整型的Tensor
- 传递给传递给torch.IntTensor的参数可以是列表，也可以是一个维度值。

```
1 import torch
2
3 a = torch.IntTensor(2, 3)
4 b = torch.IntTensor([2, 3, 4, 5])
5 print(a)
6 print(b)
```

```
tensor([[0, 0, 0],
        [0, 0, 0]], dtype=torch.int32)
tensor([2, 3, 4, 5], dtype=torch.int32)

进程已结束,退出代码0
```

■ torch.rand(size)

- 功能：生成随机数
- 输入size为要生成的张量的大小

```
1 import torch
2
3 a = torch.rand(2, 3)
4 print(a)
```

```
tensor([[0.4588, 0.5060, 0.6614],
        [0.5143, 0.8078, 0.0695]])
```

进程已结束,退出代码0

■ torch.randn(size)

- 功能：用于生成数据类型为浮点数且维度指定的随机Tensor
- 输入size为要生成的张量的大小，随机生成的浮点数的取值满足均值为0，方差为1的正态分布。

```
1 import torch
2
3 a = torch.randn(2, 3)
4 print(a)
```

```
tensor([[ 1.3945,  1.2490, -0.0287],
        [-0.1881,  0.4865, -0.9496]])
```

进程已结束,退出代码0

■ torch.arange(start, end, step) (原torch.range)

- 功能：用于生成数据类型为浮点型且起始范围和结束范围的Tensor
- start: 起始值
- end: 结束值
- step: 步长，用于指定从起始值到结束值得每步的数据间隔

```
1 import torch
2
3 a = torch.arange(1, 20, 2)
4 print(a)
```

```
tensor([ 1,  3,  5,  7,  9, 11, 13, 15, 17, 19])
```

```
进程已结束,退出代码0
```



■ torch.zeros(size)

- 功能：生成数据类型为浮点型且维度指定的Tensor，其中的元素值全部为0。
- 输入size为要生成的张量的大小

```
1 import torch
2
3 a = torch.zeros(2, 3)
4 print(a)
```

```
tensor([[0., 0., 0.],
        [0., 0., 0.]])
```

进程已结束,退出代码0



■ torch.ones(size)

- 功能： torch.ones生成全1的数组。
- 输入size为要生成的张量的大小

```
1 import torch
2
3 a = torch.ones(2, 3)
4 print(a)
```

```
tensor([[1., 1., 1.],
        [1., 1., 1.]])
```

进程已结束,退出代码0

■ torch.empty (size)

- 功能：创建一个未被初始化数值的tensor
- 输入size为要生成的张量的大小，这里可以是一个list 也可以是一个tuple

```
1 import torch
2
3 a = torch.empty(2, 3)
4 print(a)
```

```
tensor([[ -1.2466e+35,  2.0151e-42,  0.0000e+00],
        [ 0.0000e+00,  0.0000e+00,  0.0000e+00]])
```

进程已结束,退出代码0



Tensor的运算



■ torch.add(x1, x2)

- 功能：求x1和x2的求和结果
- 输入参数既可以全部是Tensor数据类型的变量，也可以一个是Tensor数据类型的变量，另一个是标量。

```
1 import torch
2
3 a = torch.tensor([[1, 2, 3, 4],
4                   [5, 6, 7, 8]])
5 print(a)
6 b = torch.ones(2, 4)
7 print(b)
8 c = torch.add(a, b)
9 print(c)
```

```
tensor([[1, 2, 3, 4],
        [5, 6, 7, 8]])
tensor([[1., 1., 1., 1.],
        [1., 1., 1., 1.]])
tensor([[2., 3., 4., 5.],
        [6., 7., 8., 9.]])
```

进程已结束,退出代码0



■ torch.mul (x1, x2)

- 功能：x1和x2进行对应元素相乘
- x1和x2的shape必须相同

```
1 import torch
2
3 a = torch.tensor([[1, 2, 3, 4],
4                   [5, 6, 7, 8]])
5 print(a)
6 b = torch.ones(2, 4) * 2
7 print(b)
8 c = torch.mul(a, b)
9 print(c)
```

```
tensor([[1, 2, 3, 4],
        [5, 6, 7, 8]])
tensor([[2., 2., 2., 2.],
        [2., 2., 2., 2.]])
tensor([[ 2.,  4.,  6.,  8.],
        [10., 12., 14., 16.]])
```

进程已结束,退出代码0



■ torch.mm (x1, x2)

- 功能：x1和x2矩阵相乘
- x1的行数必须和x2列数相等

```
1 import torch
2
3 a = torch.tensor([[1, 1, 1, 1],
4                   [2, 2, 2, 2]])
5 print(a)
6 b = torch.tensor([[1, 2],
7                   [1, 2],
8                   [1, 2],
9                   [1, 2]])
10 print(b)
11 c = torch.mm(a, b)
12 print(c)
```

```
tensor([[1, 1, 1, 1],
        [2, 2, 2, 2]])
tensor([[1, 2],
        [1, 2],
        [1, 2],
        [1, 2]])
tensor([[ 4,  8],
        [ 8, 16]])
```

进程已结束,退出代码0



■ torch.dot (x1, x2)

- 功能：x1和x2向量相乘求和
- x1和x2必须是一维向量

```
1  import torch
2
3  a = torch.tensor([1, 2])
4  print(a)
5  b = torch.tensor([3, 4])
6  print(b)
7  c = torch.dot(a, b)
8  print(c)
```

```
tensor([1, 2])
```

```
tensor([3, 4])
```

```
tensor(11)
```

```
进程已结束,退出代码0
```



■ torch.mv (x1, x2)

- 功能：求矩阵x1和向量x2的乘积
- 运用矩阵与向量之间的乘法规则进行计算，x1为矩阵，x2为向量，循序不能颠倒。

```
1  import torch
2
3  a = torch.tensor([[1, 1, 1, 1],
4                    [2, 2, 2, 2]])
5  print(a)
6  b = torch.tensor([2, 2, 2, 2])
7  print(b)
8  c = torch.mv(a, b)
9  print(c)
```

```
tensor([[1, 1, 1, 1],
        [2, 2, 2, 2]])
tensor([2, 2, 2, 2])
tensor([ 8, 16])
```

进程已结束,退出代码0



■ torch.div (x1, x2)

- 功能：求x1和x2的商
- 参与运算的参数可以全部是Tensor数据类型的变量，也可以是Tensor数据类型的变量和标量的组合。

```
1 import torch
2
3 a = torch.tensor([[1, 2, 3, 4],
4                   [5, 6, 7, 8]])
5 print(a)
6 b = torch.ones(2, 4) * 2
7 print(b)
8 c = torch.div(a, b)
9 print(c)
```

```
tensor([[1, 2, 3, 4],
        [5, 6, 7, 8]])
tensor([[2., 2., 2., 2.],
        [2., 2., 2., 2.]])
tensor([[0.5000, 1.0000, 1.5000, 2.0000],
        [2.5000, 3.0000, 3.5000, 4.0000]])
```

进程已结束,退出代码0



■ torch.pow(X, n)

- 功能：求X的n次幂
- 参与运算的参数可以全部是Tensor数据类型的变量，也可以是Tensor数据类型的变量和标量的组合。

```
1  import torch
2
3  a = torch.tensor([[1, 1, 1, 1],
4                    [2, 2, 2, 2]])
5  print(a)
6  b = torch.pow(a, 2)
7  print(b)
```

```
tensor([[1, 1, 1, 1],
        [2, 2, 2, 2]])
tensor([[1, 1, 1, 1],
        [4, 4, 4, 4]])
```

进程已结束,退出代码0



常见标准算术运算符 (+, -, *, /, **) 都可以按元素运算

```
1 import torch
2
3 a = torch.tensor([1, 2, 3, 4])
4 print(a)
5 b = torch.tensor([2, 2, 2, 2])
6 print(b)
7 print(a + b)
8 print(a - b)
9 print(a * b)
10 print(a / b)
11 print(a ** b)
```

```
tensor([1, 2, 3, 4])
tensor([2, 2, 2, 2])
tensor([3, 4, 5, 6])
tensor([-1,  0,  1,  2])
tensor([2, 4, 6, 8])
tensor([0.5000, 1.0000, 1.5000, 2.0000])
tensor([ 1,  4,  9, 16])
```

进程已结束,退出代码0

■ torch.abs(x)

- 功能：求绝对值
- 输入参数 x，输出x的绝对值，其中输入参数x必须是一个Tensor数据类型

```
1 import torch
2
3 a = torch.randn(2, 3)
4 print(a)
5 b = torch.abs(a)
6 print(b)
```

```
tensor([[ -0.7069, -1.0031, -0.2408],
        [-0.5053,  0.0779, -0.4389]])
tensor([[ 0.7069,  1.0031,  0.2408],
        [ 0.5053,  0.0779,  0.4389]])
```

进程已结束,退出代码0



■ torch.clamp(input, min, max)

- 功能：将输入Tensor 的每个元素值限制在 [min, max]区间范围，若超出该范围，则进行裁剪。输出一个裁剪后的Tensor
- input: 需要进行裁剪的Tensor数据类型的变量
- min: 裁剪的上边界
- max: 裁剪的下边界

```
y_i = | min, if x_i < min  
      | x_i, if min <= x_i <= max  
      | max, if x_i > max
```

x_i : 输入Tensor的每个元素
 y_i : 对 x_i 裁剪后的输出结果



■ torch.clamp(input, min, max)

- 功能：将输入Tensor 的每个元素值限制在 [min, max]区间范围，若超出该范围，则进行裁剪。输出一个裁剪后的Tensor
- input: 需要进行裁剪的Tensor数据类型的变量
- min: 裁剪的上边界
- max: 裁剪的下边界

```
1 import torch
2
3 a = torch.randn(2, 3)
4 print(a)
5 b = torch.clamp(a, -0.1, 0.1)
6 print(b)
```

待裁剪的Tensor a

```
tensor([[ 0.7153, -0.9317, -0.9738],
        [-0.0905, -0.3877, -1.2016]])
tensor([[ 0.1000, -0.1000, -0.1000],
        [-0.0905, -0.1000, -0.1000]])
```

裁剪后的结果 b

进程已结束,退出代码0

把a中的每个元素值都限制在[-0.1, 0.1]区间



■ 逻辑运算 ==

```
1 import torch
2
3 a = torch.tensor([[1, 2, 3, 4],
4                   [5, 6, 7, 8]])
5 b = torch.tensor([[2, 2, 2, 2],
6                   [2, 2, 2, 2]])
7 print(a == b)
```

```
tensor([[False,  True, False, False],
        [False, False, False, False]])
```

进程已结束,退出代码0



■ x.sum(*axis, *keepdims)

- 功能：对张量x所有元素求和
- 可选参数dim可设定对相应的维度求和

```
1 import torch
2
3 a = torch.tensor([[1, 2, 3, 4],
4                   [5, 6, 7, 8]])
5 print(a.sum())
```

tensor(36)

进程已结束,退出代码0



■ x.sum(*axis, *keepdims)

- 可选参数keepdims可以使轴数（维度）不变

```
1 import torch
2
3 a = torch.arange(12, dtype=torch.float32).reshape(3, 4)
4 b = a.sum(axis=0, keepdims=True)
5 print(b)
6 c = a.sum(axis=0)
7 print(c)
```

```
tensor([[12., 15., 18., 21.]]) → 2D
tensor([12., 15., 18., 21.]) → 1D
```

进程已结束,退出代码0



■ x.mean(*axis)

- 功能：求x的均值
- 注意元素类型为浮点型，可选参数axis可指定求均值维度

```
1 import torch
2
3 a = torch.arange(12, dtype=torch.float32).reshape(3, 4)
4 print(a)
5 print(a.mean())
6 print(a.mean(axis=0))
```

```
tensor([[ 0.,  1.,  2.,  3.],
        [ 4.,  5.,  6.,  7.],
        [ 8.,  9., 10., 11.]])
tensor(5.5000)
tensor([4., 5., 6., 7.])
```



Tensor的位置及访问

Tensor的访问

一个元素: [1, 2]

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

一行: [1, :]

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

一列: [1, :]

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

子区域: [1:3, 1:]

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

子区域: [::3, ::2]

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16



■ x.shape

- 可以通过shape访问张量的大小

```
1 import torch
2
3 a = torch.randn(2, 3)
4 print(a.shape)
```

```
torch.Size([2, 3])
```

进程已结束,退出代码0



■ x.numel()

- 可以通过numel(number of elements)访问张量的元素总数

```
1 import torch
2
3 a = torch.randn(2, 3)
4 print(a.numel())
```

6

进程已结束,退出代码0



■ x.reshape(size)

- 功能：改变一个张量的形状而不改变元素数量和元素值
- size: 要改变的新的尺寸

```
1  import torch
2
3  a = torch.randn(2, 3)
4  b = a.reshape(6)
5  c = a.reshape(3, 2)
6  print(b.shape)
7  print(c.shape)
```

```
torch.Size([6])
torch.Size([3, 2])
```

进程已结束,退出代码0



■ torch.cat((x1, x2, ...), *dim)

- 功能：把多个张量连结在一起
- 输入参数第一项为要连结的多个张量，可选参数dim为要连结的维度方向

```
1 import torch
2
3 a = torch.tensor([[1, 2, 3, 4],
4                   [5, 6, 7, 8]])
5 b = torch.tensor([[2, 2, 2, 2],
6                   [2, 2, 2, 2]])
7 c = torch.cat((a, b), dim=0)
8 print(c)
9 d = torch.cat((a, b), dim=1)
10 print(d)
```

```
tensor([[1, 2, 3, 4],
        [5, 6, 7, 8],
        [2, 2, 2, 2],
        [2, 2, 2, 2]])
tensor([[1, 2, 3, 4, 2, 2, 2, 2],
        [5, 6, 7, 8, 2, 2, 2, 2]])
```

进程已结束,退出代码0



■ 广播机制

即使形状不同，我们依然可以通过广播机制执行按元素操作

1. 通过适当复制元素来扩展一个或两个数组，以便在转换之后，两个张量具有相同的形状；
2. 对生成的数组执行按元素操作。

```
1 import torch
2
3 a = torch.arange(3).reshape((3, 1))
4 b = torch.arange(2).reshape((1, 2))
5 print(a)
6 print(b)
7 print(a + b)
```

```
tensor([[0],
        [1],
        [2]])
tensor([[0, 1]])
tensor([[0, 1],
        [1, 2],
        [2, 3]])
```

a和b分别是 3×1 和 1×2 矩阵，广播机制先将两个矩阵广播为 3×2 矩阵，即复制a的列，复制b的行，然后再按元素相加



■ 元素索引

- 可以用[-1]访问最后一个元素

```
1 import torch
2
3 a = torch.tensor([[1, 2, 3, 4],
4                   [5, 6, 7, 8],
5                   [9, 10, 11, 12]])
6 print(a[-1])
```

```
tensor([ 9, 10, 11, 12])
```

进程已结束,退出代码0



■ 元素索引

- 通过指定索引修改矩阵

```
1 import torch
2
3 a = torch.tensor([[1, 2, 3, 4],
4                   [5, 6, 7, 8],
5                   [9, 10, 11, 12]])
6 a[1, 2] = 99
7 print(a)
```

```
tensor([[ 1,  2,  3,  4],
        [ 5,  6, 99,  8],
        [ 9, 10, 11, 12]])
```

进程已结束,退出代码0



■ 元素索引

- 通过指定索引修改矩阵

```
1 import torch
2
3 a = torch.tensor([[1, 2, 3, 4],
4                  [5, 6, 7, 8],
5                  [9, 10, 11, 12]])
6 a[0:2, :] = 99
7 print(a)
```

```
tensor([[99, 99, 99, 99],
        [99, 99, 99, 99],
        [ 9, 10, 11, 12]])
```

进程已结束,退出代码0



■ x.T

- 功能：求矩阵x的转置

```
1 import torch
2
3 a = torch.tensor([[1, 2, 3, 4],
4                   [5, 6, 7, 8],
5                   [9, 10, 11, 12]])
6 print(a.T)
```

```
tensor([[ 1,  5,  9],
        [ 2,  6, 10],
        [ 3,  7, 11],
        [ 4,  8, 12]])
```

进程已结束,退出代码0



■ x.clone()

- 功能：通过clone()分配新的内存，复制x得到一个新的张量

```
1 import torch
2
3 a = torch.arange(12).reshape(3, 4)
4 b = a.clone()
5 b[1, 2] = 99
6 print(a)
7 print(b)
```

```
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
tensor([[ 0,  1,  2,  3],
        [ 4,  5, 99,  7],
        [ 8,  9, 10, 11]])
```

进程已结束,退出代码0

请按以下步骤进行操作：

1. 初始化一个(2, 3)的张量A，使其充满1
2. 初始化一个(2, 3)的张量B = [[0, 1, 2], [3, 4, 10]]
3. 计算C = B-A
4. 将张量C转换成(1, 6)形状的1D张量
5. 通过索引获取这个张量的第4个元素的值
6. 计算这个张量的L1范数
7. 计算这个张量的L2范数

L_1 范数，它表示为向量元素的绝对值之和：

$$\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$$

L_2 范数是向量元素平方和的平方根：

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$$



```
1  import torch
2
3  A = torch.ones(2, 3)
4  B = torch.arange(6).reshape(2, 3)
5  B[1][2] = 10
6  C = B - A
7  C = C.reshape(1, 6)
8  print(C[0][3])
9  L1 = torch.abs(C).sum()
10 print(L1)
11 L2 = torch.pow((torch.pow(C, 2).sum() / C.numel()), 0.5)
12 print(L2)
```